TryHackMe – Overpass

Penetration Testing Walkthrough Report

Prepared by: Manoj Bora
Platform: TryHackMe | Room: Overpass | Difficulty: Easy
Date: 25 September 2026

---

1. Objective

This report covers my walkthrough of the TryHackMe room "Overpass", an easy Linux box built around a fictional startup's password-manager product. The goal was to enumerate the target, find a way past the admin login, get a foothold as a low-privileged user, capture the user flag, and then escalate to root.

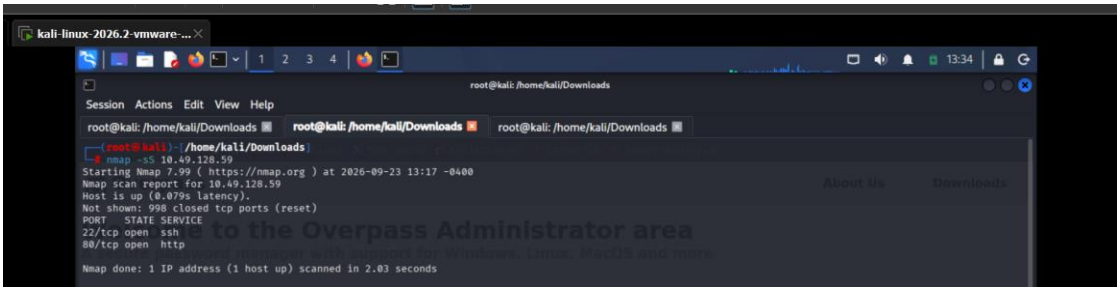
2. Tools Used

- Nmap – port scanning
- Gobuster – directory/file brute-forcing
- Firefox (view-source + DevTools Storage) – manual web inspection and cookie editing
- ssh2john – extracting a crackable hash from an SSH private key
- John the Ripper – passphrase cracking
- SSH – remote access
- crontab / sudo -l – privilege escalation enumeration
- pentestmonkey Reverse Shell Cheat Sheet – reverse shell one-liner reference
- Netcat – reverse shell listener
- Python3 http.server – hosting the malicious payload

3. Walkthrough

Reconnaissance

1. Started with an Nmap scan against the target (nmap -sS 10.49.128.59). It came back with 2 open ports — 22 (SSH) and 80 (HTTP).

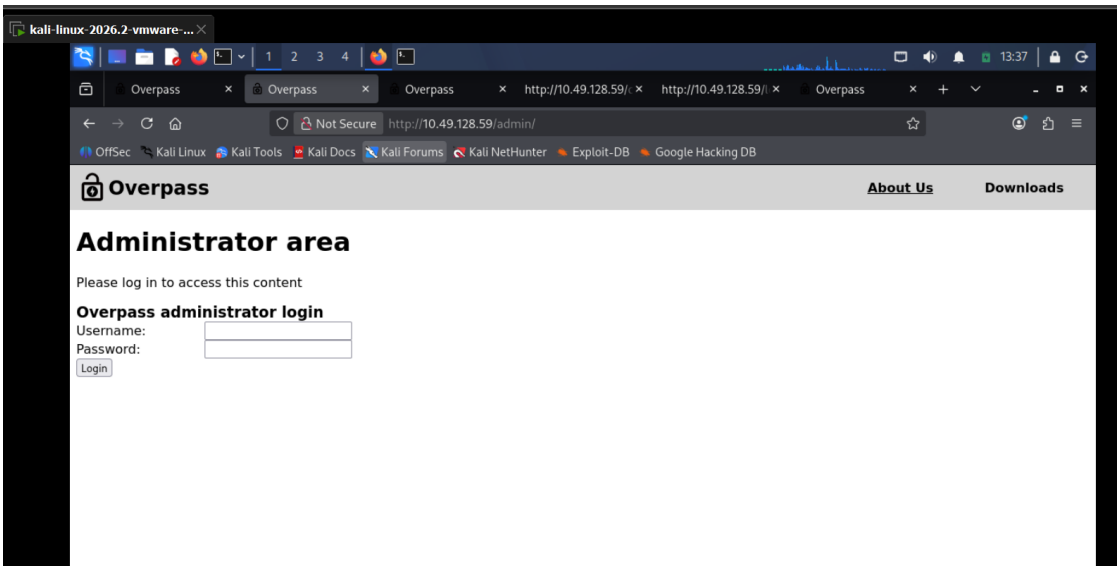


Web Enumeration

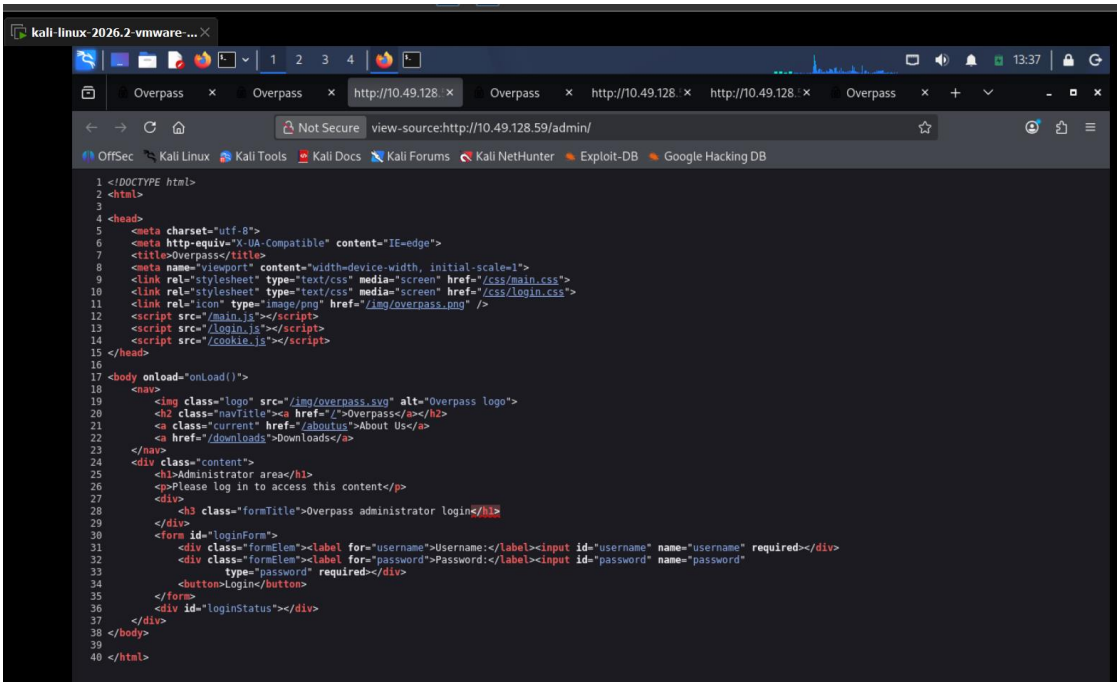
2. Since port 80 was open, ran Gobuster against the web root (gobuster dir -u http://10.49.128.59 -w directory-list-2.3-medium.txt -x php,html,txt). It turned up admin.html, an /admin/ directory, and a few others (/aboutus, /downloads, /css, /img).



3. Opened /admin/ in the browser — an "Overpass administrator login" page asking for a username and password.



4. Checked the page source of /admin/ (view-source) and found it loads two scripts, login.js and cookie.js.



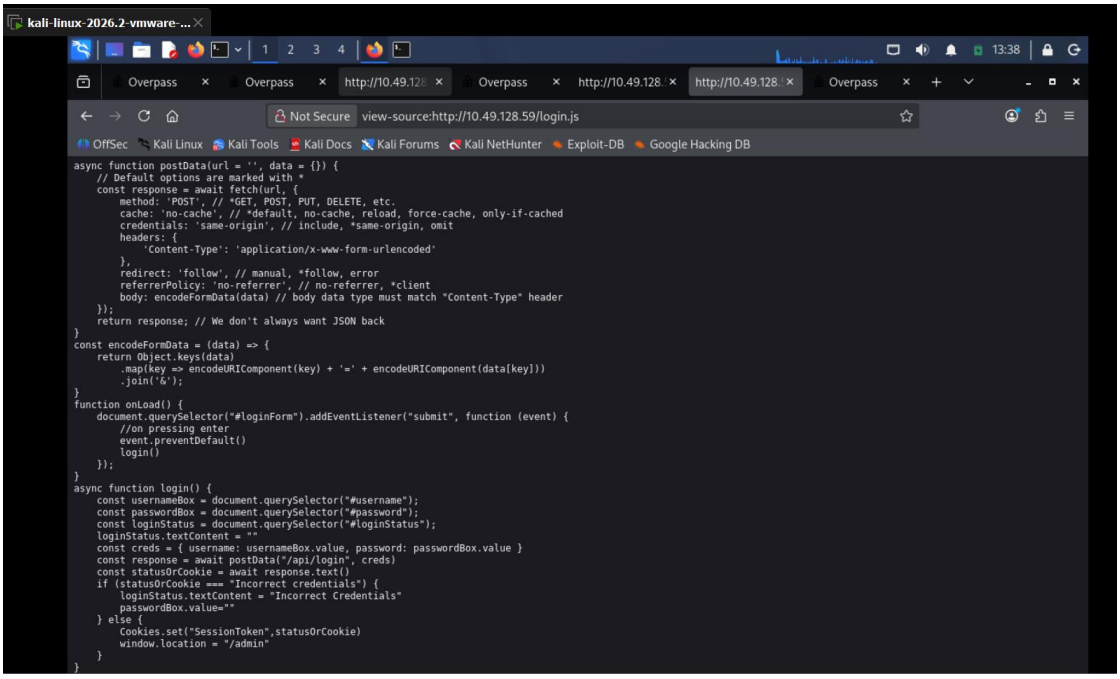

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title>Overpass</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="stylesheet" type="text/css" media="screen" href="/css/main.css">
    <link rel="stylesheet" type="text/css" media="screen" href="/css/login.css">
    <link rel="icon" type="image/png" href="/img/overpass.png" />
    <script src="/main.js"></script>
    <script src="/login.js"></script>
    <script src="/cookie.js"></script>
</head>

<body onload="onLoad()">
    <nav>
        <img class="logo" src="/img/overpass.svg" alt="Overpass logo">
        <h2 class="navTitle"><a href="/">Overpass</a></h2>
        <a class="current" href="/aboutus">About Us</a>
        <a href="/downloads">Downloads</a>
    </nav>
    <div class="content">
        <h1>Administrator area</h1>
        <p>Please log in to access this content</p>
        <div>
            <h3 class="formTitle">Overpass administrator login</h3>
        </div>
        <form id="loginForm">
            <div class="formElem"><label for="username">Username:</label><input id="username" name="username" required></div>
            <div class="formElem"><label for="password">Password:</label><input id="password" name="password"
                    type="password" required></div>
            <button>Login</button>
        </form>
        <div id="loginStatus"></div>
    </div>
</body>

</html>
```

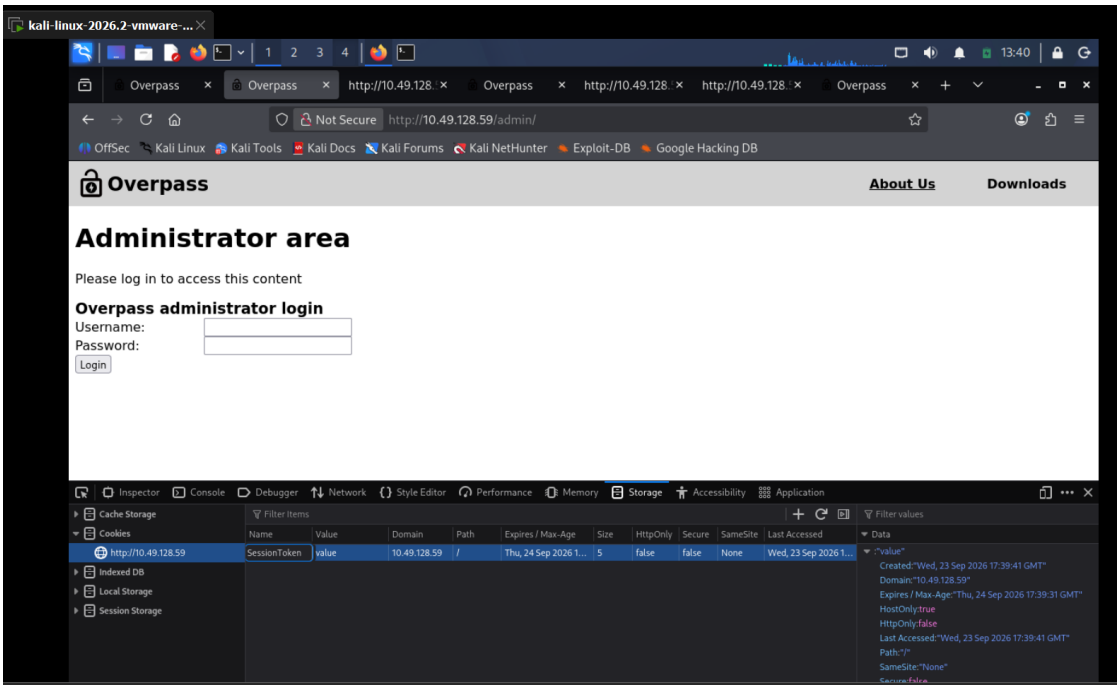
5. Read login.js. It posts the entered credentials to /api/login, and if the response text isn't "Incorrect credentials" it just takes whatever comes back, stores it in a cookie called SessionToken, and redirects to /admin — there's no real validation of the cookie's value.



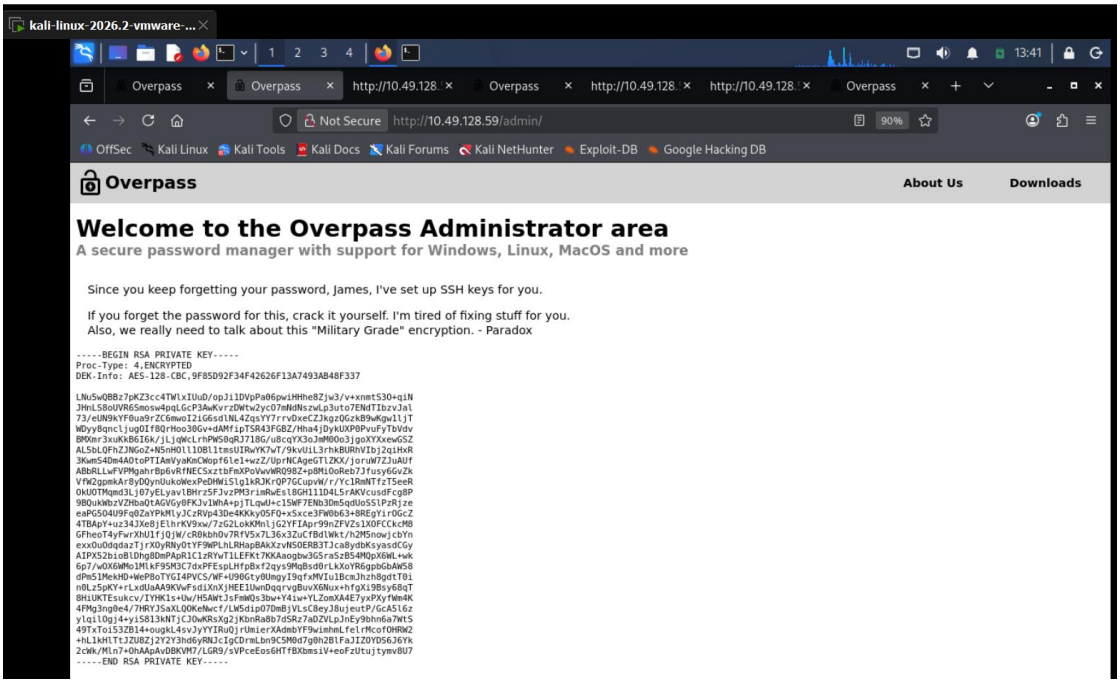

```
async function postData(url = '', data = {}) {
    // Default options are marked with *
    const response = await fetch(url, {
        method: 'POST', // *GET, POST, PUT, DELETE, etc.
        cache: 'no-cache', // *default, no-cache, reload, force-cache, only-if-cached
        credentials: 'same-origin', // include, *same-origin, omit
        headers: {
            'Content-Type': 'application/x-www-form-urlencoded'
        },
        redirect: 'follow', // manual, *follow, error
        referrerPolicy: 'no-referrer', // no-referrer, *client
        body: encodeFormData(data) // body data type must match "Content-Type" header
    });
    return response; // We don't always want JSON back
}
const encodeFormData = (data) => {
    return Object.keys(data)
        .map(key => encodeURIComponent(key) + '=' + encodeURIComponent(data[key]))
        .join('&');
}
function onLoad() {
    document.querySelector("#loginForm").addEventListener("submit", function (event) {
        //on pressing enter
        event.preventDefault()
        login()
    });
}
async function login() {
    const usernameBox = document.querySelector("#username");
    const passwordBox = document.querySelector("#password");
    const loginStatus = document.querySelector("#loginStatus");
    loginStatus.textContent = ""
    const creds = { username: usernameBox.value, password: passwordBox.value }
    const response = await postData("/api/login", creds)
    const statusOrCookie = await response.text()
    if (statusOrCookie === "Incorrect credentials") {
        loginStatus.textContent = "Incorrect Credentials"
        passwordBox.value=""
    } else {
        Cookies.set("SessionToken",statusOrCookie)
        window.location = "/admin"
    }
}
```

Authentication Bypass

6. Since the app only checks that a SessionToken cookie exists rather than validating it, created one manually via Firefox DevTools' Storage tab (name: SessionToken, value: "value") and reloaded /admin/.

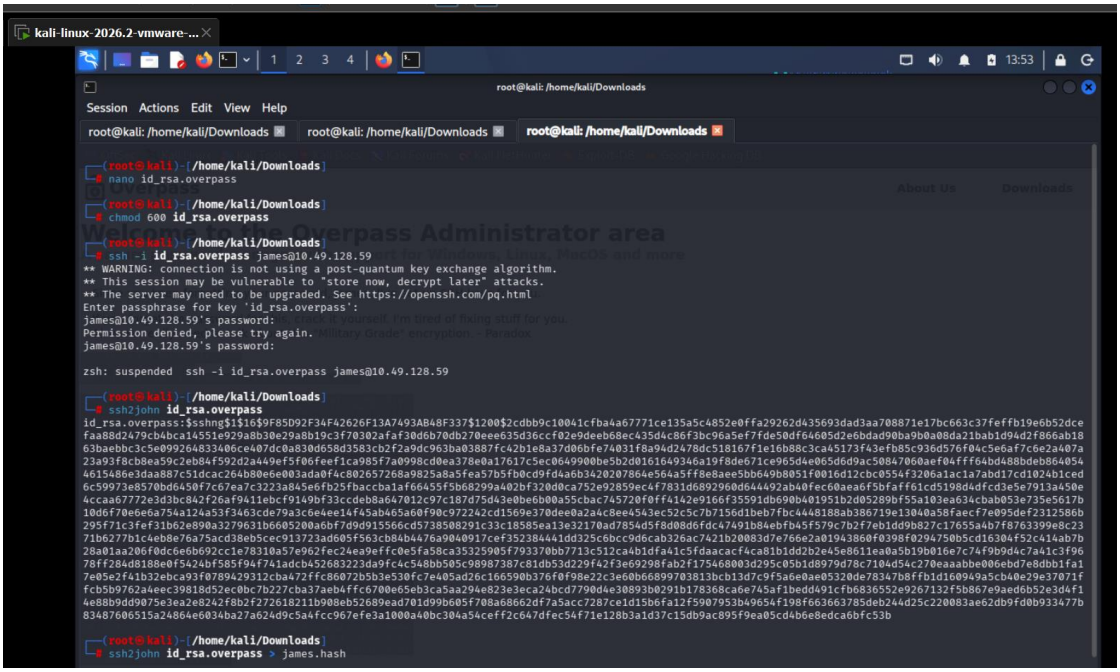


7. That was enough to get into the admin area. The page welcomed James with a note from a colleague ("Paradox") saying SSH keys had been set up for him, followed by an encrypted RSA private key pasted directly on the page.

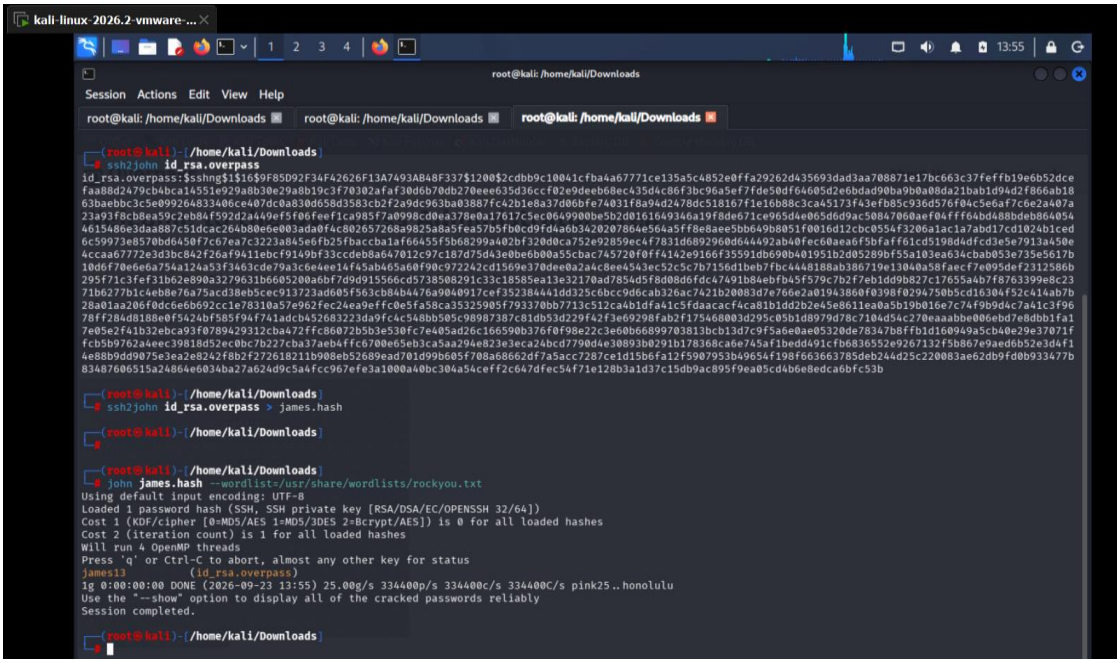


Cracking the SSH Key

8. Saved the key as id_rsa.overpass, set permissions with chmod 600, and tried to SSH in — but the key needed a passphrase. Ran ssh2john against it to pull out a crackable hash and saved it to james.hash.

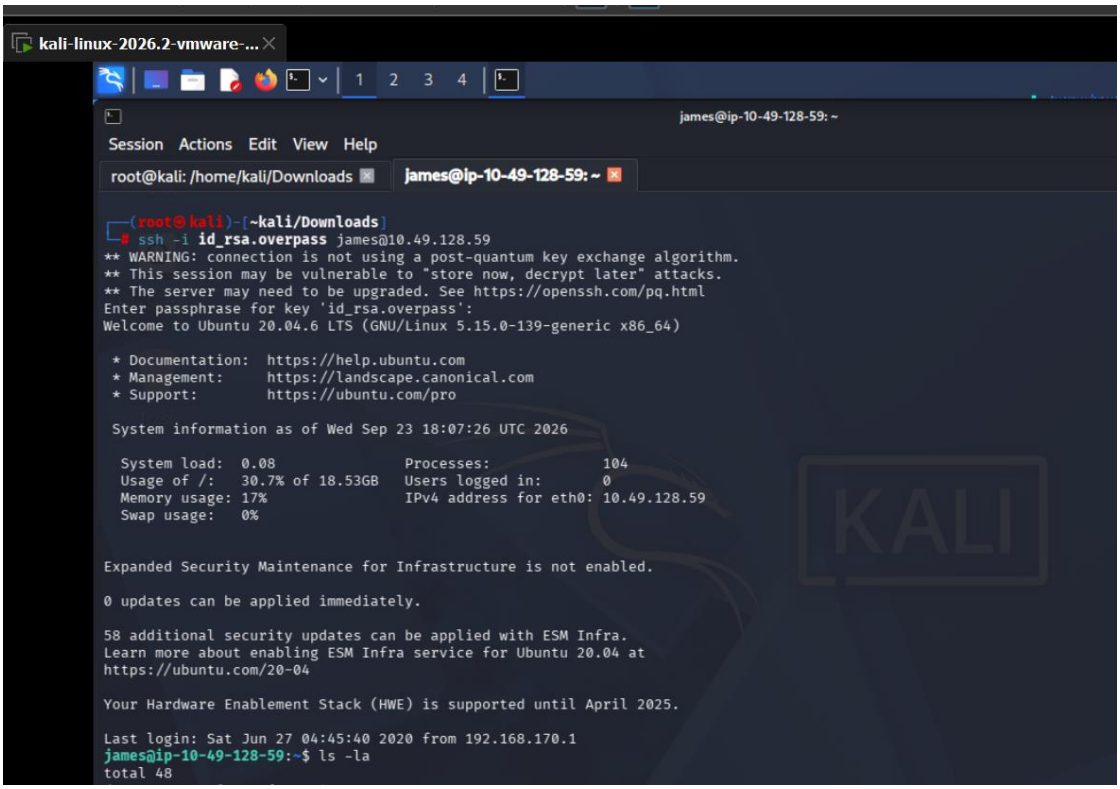


9. Cracked james.hash with John the Ripper using the rockyou.txt wordlist. It recovered the passphrase james13 almost instantly.

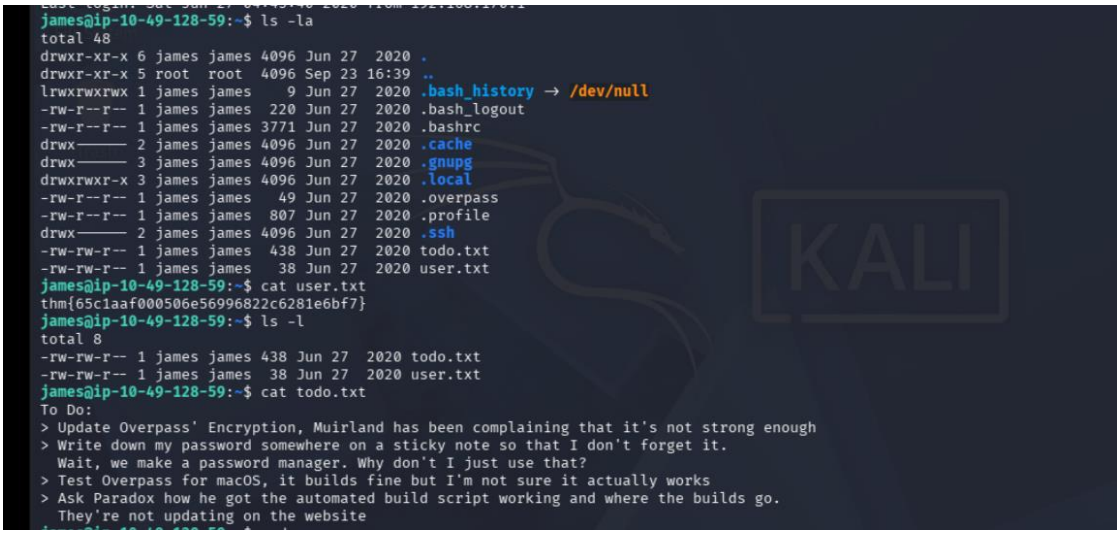


Foothold

10. Logged in over SSH with the key and the cracked passphrase (ssh -i id_rsa.overpass james@10.49.128.59) and landed on an Ubuntu 20.04.6 LTS shell as james.

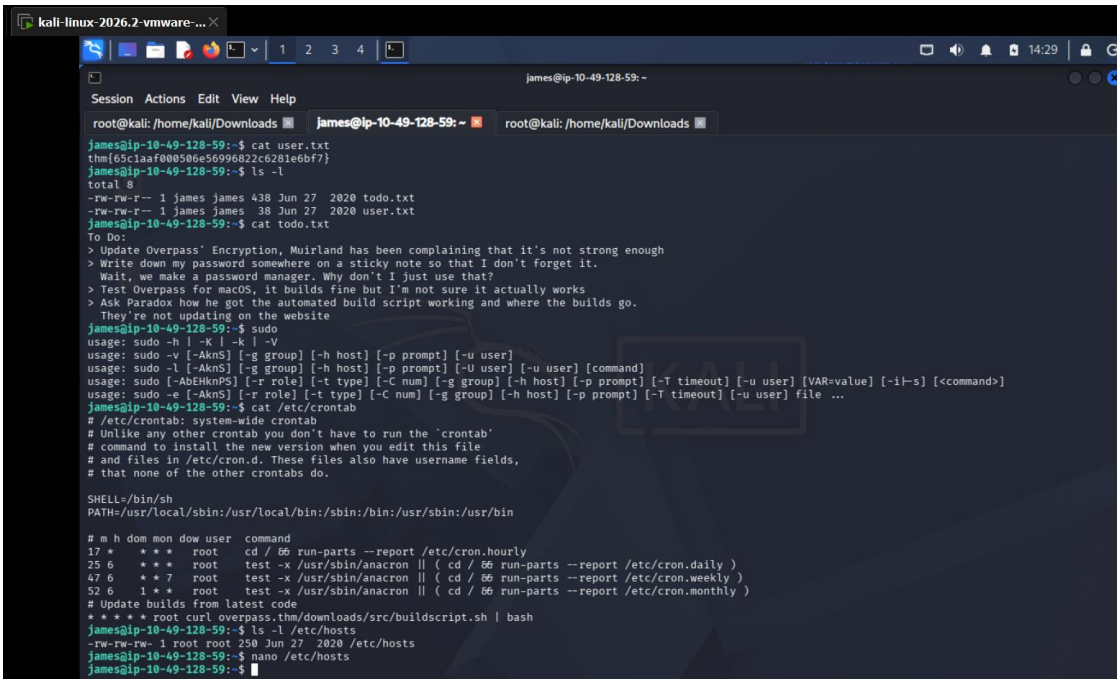


11. Read user.txt for the user flag, then checked todo.txt — it mentioned Overpass' weak encryption, an automated build script, and a note asking "Paradox" how the build script works and where the builds go. A strong hint for privilege escalation.

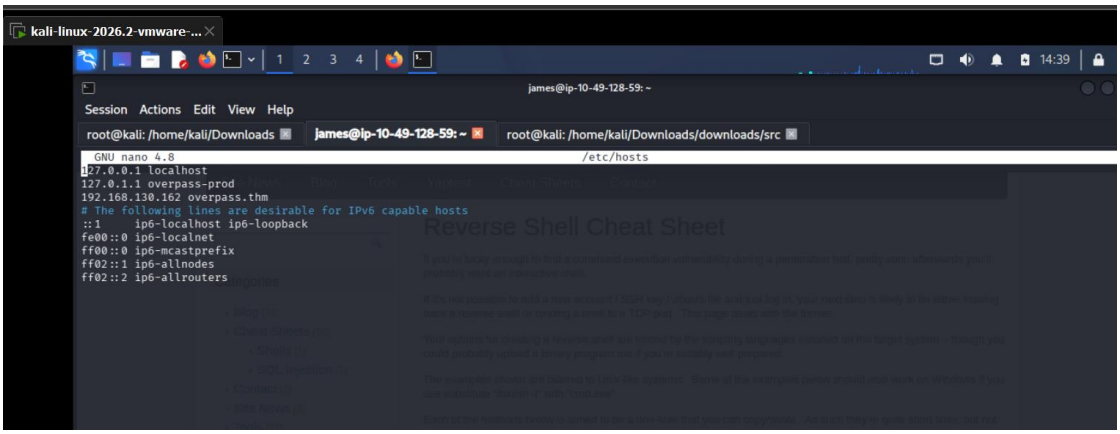


Privilege Escalation Enumeration

12. sudo alone showed no usable permissions, so reviewed /etc/crontab instead. It revealed a root cron job running every minute: curl overpass.thm/downloads/src/buildscript.sh | bash. Checked the permissions on /etc/hosts and found it was world-writable.



13. Opened /etc/hosts in nano and added an entry pointing overpass.thm at my Kali box's IP — since the file is writable by anyone, root's cron job would now curl whatever I chose to host at that domain.

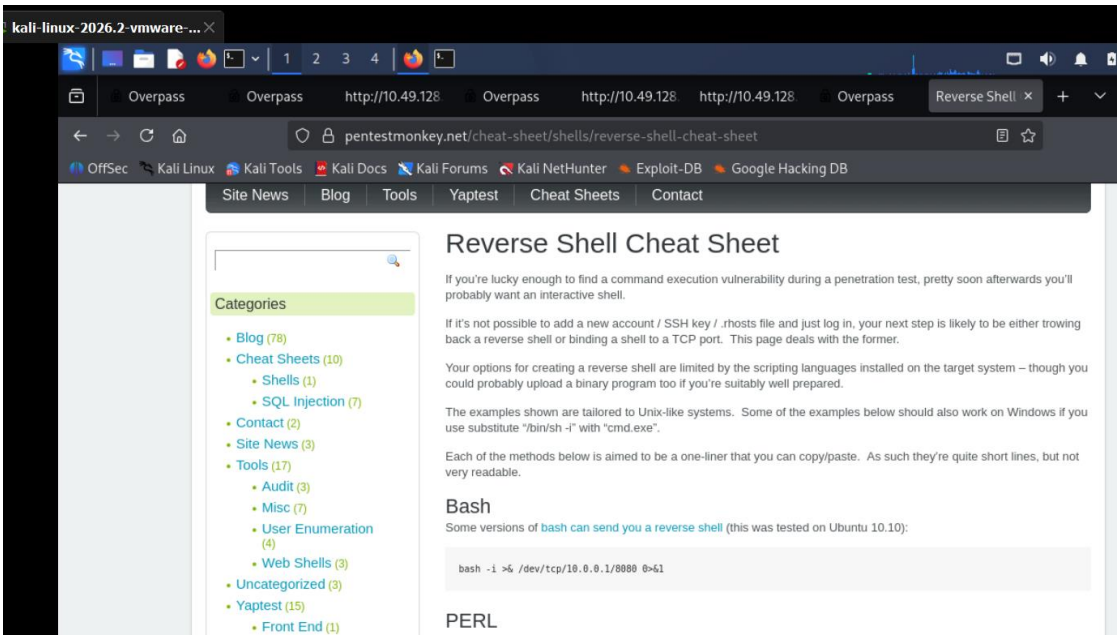


Privilege Escalation

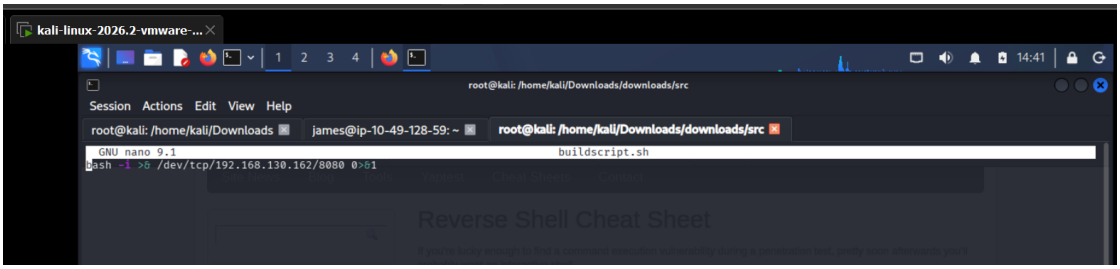
14. On Kali, recreated the exact folder path the cron job requests (downloads/src/buildscript.sh) so the served file would line up with the URL root's cron expects.



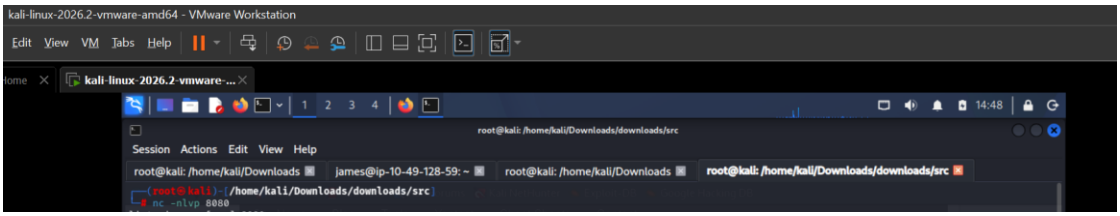
15. Pulled up pentestmonkey's Reverse Shell Cheat Sheet for a ready-made bash one-liner to drop into the script.



16. Wrote that one-liner into buildscript.sh, pointing it back at my Kali IP on port 8080.



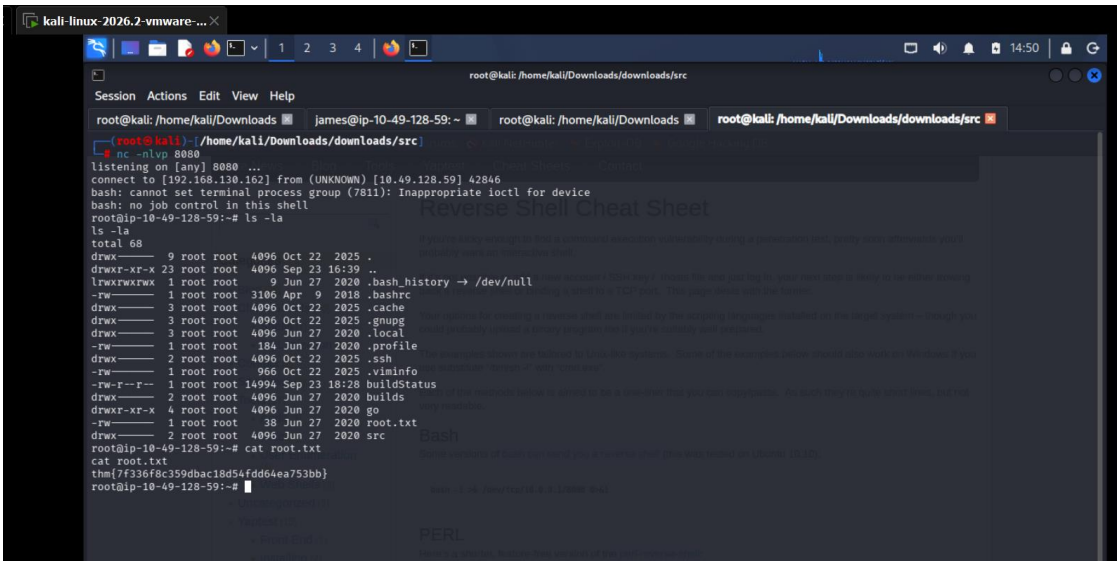
17. Started a Netcat listener on port 8080 to catch the callback.



18. Served the folder over HTTP with python3 -m http.server 80. The access log confirmed root's cron job was pulling buildscript.sh from the target once every minute, right on schedule.



19. A minute later the cron job executed the payload as root, and the Netcat listener caught a root shell. Read root.txt to grab the final flag.



4. Flags Captured

User Flag: `thm{65c1aaf000506e56996822c6281e6bf7}`

Root Flag: `thm{7f336f8c359dbac18d54fdd64ea753bb}`

5. Findings & Vulnerabilities

Summary of the weaknesses that made this box solvable, with a quick fix for each:

Finding	Severity	Suggested Fix
Directory brute-forcing exposed the admin panel and internal paths with no access restriction	**Low**	Restrict or rate-limit directory listing; don't rely on an unlisted path as the only protection for an admin login
Login page only checks that a SessionToken cookie exists, not that its value is valid — a full authentication bypass	**Critical**	Validate sessions server-side on every request using signed or server-issued tokens checked against a real session store
Admin page exposed a private SSH key protected only by a weak, wordlist-crackable passphrase	**High**	Never expose private keys through a web app; if unavoidable, use a strong unique passphrase and rotate the key
Root's crontab curls and executes a remote build script every minute over plain HTTP, and /etc/hosts is world-writable	**Critical**	Serve build scripts over HTTPS with checksum/signature verification, pin the source IP, and lock /etc/hosts to root-only write access

6. Conclusion

Overpass was a solid beginner box built on a chain of small trust failures rather than one flashy exploit: a login page that checked for a cookie instead of validating it, a private key left exposed once past that check, and a root-owned automation job that trusted both DNS resolution and a plaintext download without question. The main takeaways were to never assume a

client-side check is enforced on the server, to treat any "curl | bash" style automation as a direct path to compromise if its source can be tampered with, and to keep files like /etc/hosts locked down to the users who actually need to write to them.